FPGA LAB
Assignment-4: Fixed-Point Division Using Newton-Raphson Method

Murra Rajesh Kumar Reddy
EE24BTECH11043

1 Introduction

This report presents the implementation of fixed-point division using the Newton-Raphson method. The division is performed by computing the reciprocal of the denominator and multiplying it by the numerator:

$$\frac{A}{D} = A \cdot \frac{1}{D}$$

The implementation is done using Q2.30 fixed-point format in both Python and Verilog.

2 Mathematical Derivation

2.1 Newton-Raphson for Reciprocal

We wish to compute:

$$f(x) = \frac{1}{x} - D$$

Newton-Raphson iteration:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Derivative:

$$f'(x) = -\frac{1}{x^2}$$

Substituting:

$$x_{k+1} = x_k - \frac{\frac{1}{x_k} - D}{-\frac{1}{x_k^2}}$$

Simplifying:

$$x_{k+1} = x_k + x_k(1 - Dx_k)$$

$$x_{k+1} = x_k(2 - Dx_k)$$

This is the recurrence used in the implementation.

2.2 Quadratic Convergence

If error at iteration k is:

$$e_k = x_k - \frac{1}{D}$$

Then:

$$e_{k+1} \approx e_k^2$$

Thus convergence is quadratic.

3 Normalization

For fast convergence, D is normalized into:

$$0.5 \leq D_{\text{norm}} < 1$$

If original denominator is D, we compute:

$$D_{\text{norm}} = D \cdot 2^{\text{shift}}$$

Then:

$$\frac{1}{D} = \frac{1}{D_{\text{norm}}} \cdot 2^{\text{shift}}$$

The final scaling accounts for this shift.

4 Fixed-Point Representation

We use Q2.30 format:

$$\text{Value} = \frac{\text{Integer}}{2^{30}}$$

Multiplication rule:

$$(a \times b)_{\text{fixed}} = \frac{a \cdot b}{2^{30}}$$

Implemented as:

$$(a \cdot b) >> 30$$

5 Smart Initial Guess Using Minimax Approximation

5.1 Objective

After normalization, the denominator satisfies:

$$0.5 \leq d < 1$$

We approximate:

$$f(d) = \frac{1}{d}$$

using a linear function:

$$x_0(d) = a - bd$$

The goal is to minimize the maximum approximation error over the interval:

$$\min_{a,b} \max_{d \in [0.5,1]} \left| \frac{1}{d} - (a - bd) \right|$$

This is known as a minimax approximation.

5.2 Error Function

Define the approximation error:

$$E(d) = \frac{1}{d} - a + bd$$

For a best minimax linear approximation, the error must attain equal magnitude and alternating sign at three points in the interval (Equioscillation Theorem).

For the interval $[0.5, 1]$, these occur at:

$$d = 0.5, \quad d = c \in (0.5, 1), \quad d = 1$$

5.3 Endpoint Condition

At the endpoints:

$$E(0.5) = 2 - a + 0.5b$$

$$E(1) = 1 - a + b$$

For equal ripple behavior:

$$E(0.5) = -E(1)$$

Substituting and simplifying gives:

$$a = \frac{3 + 1.5b}{2}$$

5.4 Interior Extremum Condition

The interior extremum satisfies:

$$\frac{dE}{dd} = 0$$

Since:

$$\frac{dE}{dd} = -\frac{1}{d^2} + b$$

At the extremum $d = c$:

$$b = \frac{1}{c^2}$$

Solving the equal-ripple system yields:

$$a = \frac{48}{17}, \quad b = \frac{32}{17}$$

5.5 Final Initial Guess

Thus, the optimal linear minimax approximation is:

$$x_0(d) = \frac{48}{17} - \frac{32}{17}d$$

Numerically:

$$x_0(d) \approx 2.823529 - 1.882353d$$

5.6 Why This is a Good Choice

- Guarantees bounded worst-case error over $[0.5, 1]$.
- Maximum relative error $\approx 3\%$.
- Requires only one multiplication and one subtraction.
- Ensures fast quadratic convergence of Newton-Raphson.

6 Algorithm Summary

1. Normalize denominator
2. Compute smart initial guess
3. Perform 5 Newton-Raphson iterations
4. Adjust scaling
5. Multiply by numerator

7 Test Cases (Prime Denominators)

$$\frac{15}{23}, \quad \frac{10}{79}, \quad \frac{7}{97}, \quad \frac{25}{53}, \quad \frac{9}{71}$$

Prime denominators avoid trivial simplifications.

8 Results

8.1 Python Implementation

```
Python Reference Results:

15/23
Fixed  : 0.6521739112213254
True   : 0.6521739130434783
Error  : 1.8221528730322234e-09

10/79
Fixed  : 0.12658227235078812
True   : 0.12658227848101267
Error  : 6.1302245502048436e-09

7/97
Fixed  : 0.07216494623571634
True   : 0.07216494845360824
Error  : 2.217891897915436e-09

25/53
Fixed  : 0.4716980969533324
True   : 0.4716981132075472
Error  : 1.6254214751931784e-08

9/71
Fixed  : 0.1267605610191822
True   : 0.1267605633802817
Error  : 2.361099482595108e-09
```

Figure 1: Python Implementation Results

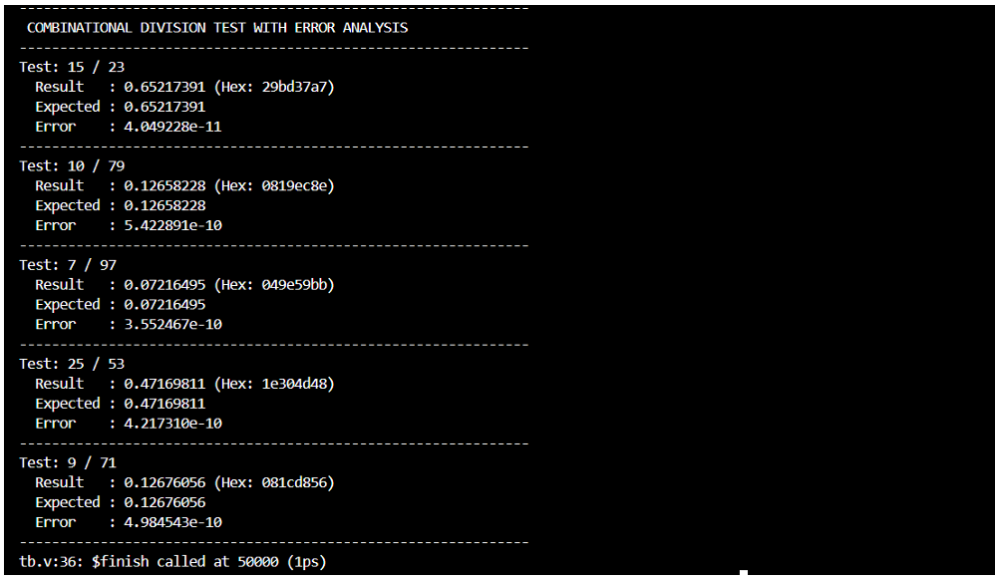

Figure 2: Verilog Results

8.2 Verilog Implementation

8.3 Observations

The Verilog simulation results closely match the Python reference results, confirming the correctness of the fixed-point hardware implementation. In the Verilog outputs, the absolute error is on the order of 10^{-10}, whereas in the Python reference it is slightly larger, typically around 10^{-9} to 10^{-8}. These differences are extremely small and indicate high numerical accuracy in both cases. The minor discrepancies arise from fixed-point quantization and rounding effects in the hardware model, compared to Python's double-precision floating-point arithmetic. Overall, the numerical agreement between the two implementations validates that the Verilog design accurately approximates the true division results with only negligible precision loss.

9 Conclusion

The Newton-Raphson method provides fast quadratic convergence for reciprocal computation. Using normalization and a smart linear initial approximation significantly reduces iteration count. The Q2.30 format ensures high precision while maintaining hardware efficiency.